

ISA 345 Final Project Report

Anna O'Connell, Aliya Merchant, Ella O'Neill, Ella Weinbauer, Josephine Timperman

ISA 345 Section C

04/29/2026

PROJECT OVERVIEW

We created this database for a pizza shop called “ISA Slice”. The shop owner, Sally, wants to create a database to boost operational efficiency. Over the years, ISA Slice has been declining in revenue and often finds itself over or under staffed depending on the day. She wants to find a way to track employees, their locations, and the orders that each employee completes. Throughout this report, we will outline an employee tracking system to better schedule employees for shifts.

ISA Slice sells both small and large pizzas, and offers five different types of pizzas: cheese, pepperoni, supreme, veggie, and meat lover’s. Sally wonders which menu items are selling the best, and which are underperforming so that they can start implementing marketing strategies and promotions. Additionally, Sally thinks that keeping track of customers' names and addresses can boost retention and wants to start a customer loyalty program, so she wants a database that will keep track of customers. Lastly, Sally often finds that workers are understaffed or overstaffed, so she wants to do a better job keeping track of employees, how many orders they handle, and the date and time of each order. Creating a database to store all employee and order information can help her answer these of questions.

The database supports several key operational transactions that reflect real-world business processes within the pizza ordering system. These transactions ensure efficient order processing, accurate data management, and improved customer service.

Example 1: A customer orders a Large Pepperoni Pizza

1. When a customer calls in a pepperoni pizza, a new record is created in the orders table: there is a unique OrderID, the current Date, and the EmployeeID who enters the order into the register.
2. Next, a record is added to the OrderDetails table for each item ordered: in this case, just one record is created for one large pepperoni pizza, with a link to the OrderID. If a customer orders multiple items, each line item has a unique OrderDetailsID but can share the same OrderID. The system pulls the current price of the item from the Menu Items table to calculate the OrderTotal.

This supports Sally’s business because the kitchen knows what to make and Sally can later track which employee is most efficiently taking orders, which items are ordered most frequently, and calculate order totals and revenues.

Example 2: Sally wants to run a business promotion for the most loyal customers

1. The system performs a JOIN between the Customers and Orders tables. It performs a SUM() function for OrderTotal by CustomerID to identify the highest value customers.
2. The database generates a list of PhoneNo and FirstName from customers.

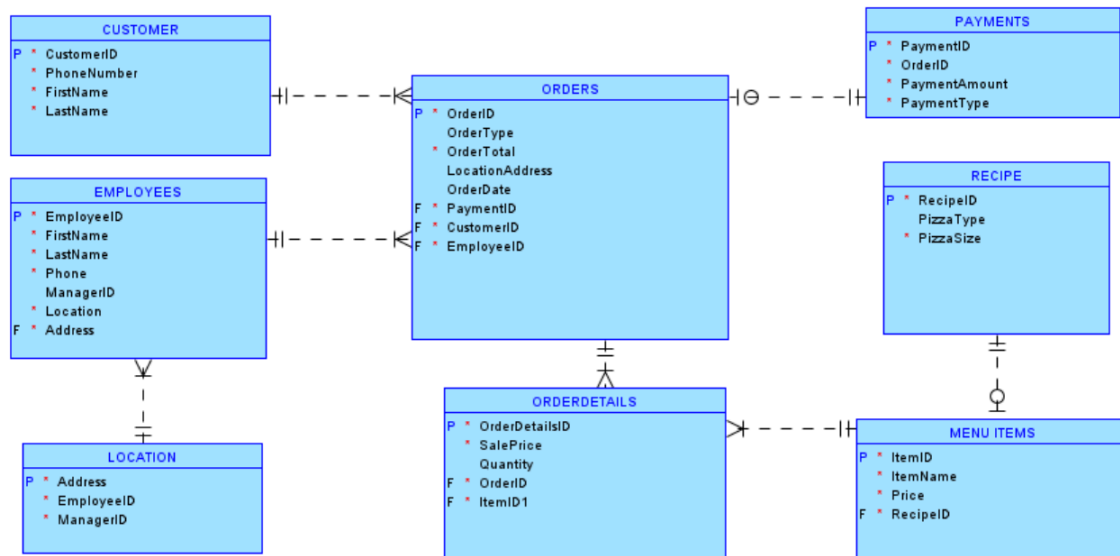
Sally is able to send out coupons to the list of phone numbers with personalized messages to bring highest spenders back into the store. She gives reminders to people who already like ISA Slice pizza, maximizing the return for sending out her coupon.

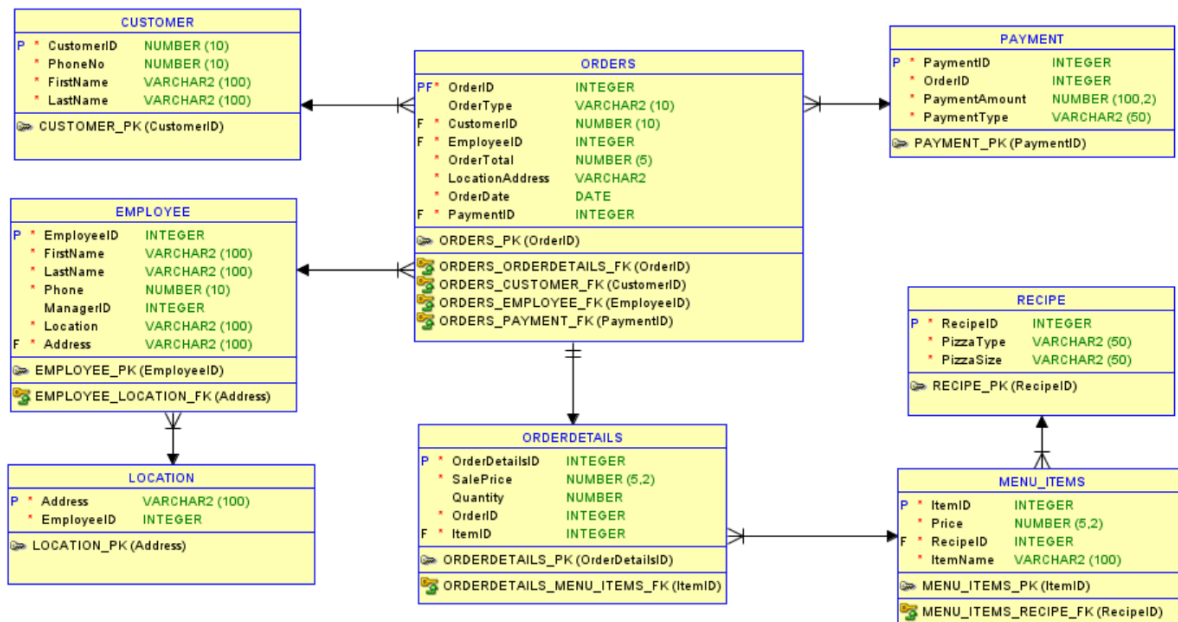
Example 3: Reassigning Employee based on Store Demand

1. After realizing that the “123 Maple Street” location has a higher demand than “456 Oak Ave” on a Friday night, Sally wants to reassign employees from the South location to help out at the North store. First, Sally performs an UPDATE on the Employees table. By using the specific EmployeeID, she changes the LocationID (Foreign Key).
2. Because the database enforces a 1:M relationship between Location and employees, the system ensures the employee is officially only on one store at a time. If an employee wants to move back, she will have to do this UPDATE again.

This supports Sally’s business by allowing her to solve over/under staffing issues by switching employees between locations as needed.

DATABASE DESIGN AND IMPLEMENTATION





We first decided as a group which entities we would need: Orders, OrderDetails, Price, Customers, Menu Items, Employees, Locations, and Recipe.

(1) ORDERS: This is the first entity we decided that we needed. This is the main information for each transaction.

Attributes: OrderID (PK), OrderType, CustomerID, EmployeeID, OrderTotal, LocationAddress, OrderDate, PaymentID

The first issue came early on when we added too many attributes before we realized we also needed an OrderDetails Table: the Orders table to store information for each transaction, and the Order Details table to store information about each line item. This means there are often multiple rows for the same OrderID, but the primary key of this table is OrderDetailsID or Line Item.

(2) ORDERDETAILS: This entity has line items of each order.

Attributes: OrderDetailsID (PK), SalePrice, Quantity, OrderID, ItemID

This clarification helped us figure out how else to build our database: once we had attributes for Orders and OrderDetails, we created other tables to store that information.

(3) CUSTOMERS: This entity stores customers and their contact information, updating each time an order is placed.

Attributes: CustomerID (PK), FirstName, LastName, PhoneNo

(4) EMPLOYEES: Tracks staff and which locations they are assigned to

Attributes: EmployeeID (PK), FirstName, LastName, Phone, ManagerID, Location, Address

(5) LOCATIONS: The different locations of ISA Slice

Attributes: Address (PK), ManagerID

(6) MENUITEMS: The list of all items on the menu of the shop and how much they cost to purchase.

Attributes: ItemID (PK), Price, RecipeID, ItemName

(7) RECIPE: This defines specific size and type of each item on the menu items list.

Attributes: RecipeID, PizzaType, PizzaSize

(8) PAYMENT: Records how each order is paid for

Attributes: PaymentID (PK), OrderID, PaymentAmount, PaymentType

Once we had all of our entities mapped out on a document, we created them on SQL logical modeler.

Next, we moved into the implementation phase and created connections in our logical model. We talked through each connection we would need.

Orders to OrderDetails: This was the first connection our group decided on. A single order has one or more line items, each line item only belongs to one OrderID. OrderID is a FK in the Order Details Table.

Orders to Customers: Every order needs to have a CustomerID to be a Foreign Key to connect to the CustomerID table. A customer can place zero or many orders, but each order must be associated with exactly one customer. This way, ISA Slice can track customers' names, phone numbers, and OrderIDs to be able to query the most loyal customers.

Orders to Employees: Additionally, every Order needs to be entered into the system by an employee, so we used an EmployeeID as a foreign key and made another 1:M connection to the Employee table. Every order can be done by one employee, but each employee has many orders.

Employees to Locations: Furthermore, this led us to decide that ISA Slice has many locations. Each employee works at exactly one location, but each location has many employees. We used Address as the Primary Key for the Location Table and as a FK for the Employees table. We also included ManagerID as an optional field in the Employee table but as a mandatory field in the Location table, because every location has one manager.

Orders to Payments: Every order has a PaymentID, a unique PK in the Payments table. This is a unique ID for each transaction. Furthermore, each Order could have two payments, so a split payment order would have two unique payment IDs.

OrderDetails to MenuItems: Each menu item can appear on many different order line items. Each line item refers to exactly one specific menu item, allowing the price to connect to order details.

MenuItems to Recipe: In order not to clutter the MenuItems table, we included a Recipe table that will store information like Pizza Type and Size.

Orders to Locations: When we were working on the report to the project, we also realized we needed a connection from Orders to location so that Sally would be able to see which location takes the most orders. We connected these takes on LocationAddress.

Finally, once we mapped out the logical model and all of the connections, we created a physical database. As a group, we talked through the best data type for each attribute.

TEST OF NORMALIZATION

We started off removing redundancy within our tables. So instead of having the same data stored in multiple tables which is a waste of resources, we use normalization to remove anomalies. Our functional database already satisfies the First Normal Form by default because all attributes are atomic and each table contains its own primary key. This is to be expected for any properly built relational database.

To evaluate the second normal form, we had to ensure that there were no partial dependencies. A partial dependency occurs when a non-key column is determined by only part of the composite primary key. In our original database design the primary key for the OrderDetails table was (OrderId, ItemID). However, through the 2NF normalization steps we realized that this would violate 2NF. The reason for this is because SalePrice depends only on ItemOD and not the entire key of (OrderId, ItemID). Therefore, we changed the OrderDetails table and added a new primary key called OrderDetailsID as the primary key to pass through 2NF.

Next, to evaluate whether or not our table is in 3NF we had to ensure that every non-key attribute depends only on the primary key and not another non-key attribute. The CUSTOMERS table has no chaining. However, in the EMPLOYEES table there is an issue with the transitive dependency of EmployeeID to Address and Address to Location Info. Therefore, we changed the Address to LocationID. Then, for the ORDERS table there is a transitive dependency between OrderID to LocationAddress to LocationDetails. The way to solve this was to create LocationID in the ORDERS Table.

OPERATIONAL SQL

Query 1: Identify the customers who place the largest orders

```
SELECT FirstName, LastName, PhoneNumber
FROM CUSTOMER
WHERE CustomerID IN (
    SELECT CustomerID
    FROM ORDERS
    WHERE OrderTotal > (SELECT AVG(OrderTotal) FROM ORDERS)
);
```

Our shop needs to know who their biggest spenders are so they can send them specific ads and coupons to keep them coming back. This query identifies customers who have placed orders with totals higher than the shop's average order value.

Query 2: Generate a receipt for a customer

```
SELECT ORDERS.OrderID, MENU.ItemName, ORDERDETAILS.Quantity,
ORDERDETAILS.SalePrice, (ORDERDETAILS.Quantity * ORDERDETAILS.SalePrice) AS
LineTotal
FROM ORDERS INNER JOIN ORDER_DETAILS ON ORDERS.OrderID =
ORDERDETAILS.OrderID INNER JOIN MENU_TABLE ON ORDERDETAILS.ItemID =
MENU.ItemID
WHERE ORDERS.OrderID = 1001;
```

This creates a receipt for a customer whenever they place an order. It would need to be altered in the WHERE line depending on which OrderID it is.

Query 3: Finds menu items that have not been ordered

```
SELECT MENU.ItemName, MENU.Price
FROM MENU_TABLE LEFT OUTER JOIN ORDER_DETAILS ON MENU.ItemID =
ORDERDETAILS.ItemID
WHERE ORDERDETAILS.OrderDetailsID IS NULL;
```

This is most important for the kitchen and the managers of the restaurant. It shows the menu items that have not yet been ordered so they can stop making it and replace it with something else.

Query 4: Analyze different payment types used after a given period of time

```
SELECT PaymentType, SUM(PaymentAmount) AS TotalRevenue, COUNT(PaymentID) AS  
NumberOfTransactions  
FROM PAYMENT  
GROUP BY PaymentType;
```

This shows the store at the end of day how many transactions were made on the different variations of how customers can pay. It counts how many transactions were done on each method.

Query 5: Identify high-performing employees

```
SELECT EMPLOYEE.FirstName, EMPLOYEE.LastName, SUM(ORDERS.OrderTotal) AS  
TotalSalesProcessed  
FROM EMPLOYEE INNER JOIN ORDERS ON EMPLOYEE.EmployeeID = ORDERS.EmployeeID  
GROUP BY EMPLOYEE.FirstName, EMPLOYEE.LastName  
HAVING SUM(ORDERS.OrderTotal) > 1000;
```

This query identifies top-performing employees (that have handled more than 10 orders), which is useful for promotions, scheduling, and general performance reviews.

INDEX:

```
CREATE INDEX IDX_CUSTOMER_PHONE ON CUSTOMER(PhoneNumber);  
CREATE INDEX IDX_ORDERS_CUSTOMER ON ORDERS(CustomerID);
```

It is common for our company to have to search for customers by their phone numbers. The INDEX prevents the database from looking through every line and instead jumps to the correct customer record.

VIEW:

```
CREATE VIEW V_DAILY_SALES AS SELECT TRUNC(OrderDate) AS SalesDate,  
COUNT(OrderID) AS TotalOrders, SUM(OrderTotal) AS TotalRevenue  
FROM ORDERS  
GROUP BY TRUNC(OrderDate);
```

Managers do not want to have to try and read complex queries and code. This VIEW creates a table that hides all of that. A manager can simply run the queries of `SELECT * FROM V_Daily_Sales` to gather a complete dashboard, hiding all of the behind the scenes information.

PROCEDURE:


```

CREATE OR REPLACE PROCEDURE Process_Refund ( PAYMENT_PaymentID IN INTEGER,
PAYMENT_OrderID IN INTEGER, PAYMENT_RefundAmount IN NUMBER,
PAYMENT_PaymentType IN VARCHAR2 )
AS BEGIN
    PAYMENT (negative amount) INSERT INTO PAYMENT (PaymentID, OrderID,
PaymentAmount, PaymentType) VALUES (PAYMENT_PaymentID, PAYMENT_OrderID,
PAYMENT_RefundAmount, PAYMENT_PaymentType);
    ORDERS UPDATE ORDERS SET OrderTotal = OrderTotal - PAYMENT_RefundAmount
WHERE OrderID = PAYMENT_OrderID;
COMMIT;
END;
/

```

If there was a refund that needed to be processed, it would require updating quite a few of the tables in the database. This leaves room for human error if a certain table is not altered correctly or forgotten completely. This procedure combines both the INSERT and the UPDATE into a single transaction

TRIGGER:

```

CREATE OR REPLACE TRIGGER TRG_ORDERS_DATE
BEFORE INSERT ON ORDERS
FOR EACH ROW
BEGIN
    IF :NEW.OrderDate IS NULL THEN :NEW.OrderDate := SYSDATE;
    END IF;
END;
/

```

Ideally you would want to be able to automate your system so you don't have to remember every single rule when using the database. This TRIGGER acts as a safety net for that. So if an employee tries to insert an order without specifying a date, the database will automatically catch it and add it with the current system date.

CONCLUSIONS AND LESSONS LEARNED

This database system is going to help the operational efficiency of ISA Slice by storing information, facilitating transactions, and allowing Sally to collect different sorts of data. Out of our original goals, we can see that this database helps Sally with staffing by tracking employee efficiency by location, Customer retention by identifying loyal and high-spending customers, and day-to-day pizza ordering, making, and selling.

This project challenged our group greatly. We had to solve many problems: at first, we had different entities for each menu item like “Pizza”, “Salad”, “Etc” until we decided we could have all menu items all on a MenuItem table with relevant prices. This caused us to restructure our database and create the “Locations” and “Payments” tables to reach the 8 entity requirement. By problem-solving as a group, we could talk through why we were making each decision, which helped boost our understanding.

Moving forward, we now know how to create a database from scratch to support business processes. Not only this, but we could brainstorm what data would be helpful to a pizza restaurant and act as a consultant for ISA Slice.

APPENDIX

A. Oracle DDL statements: CREATE TABLE statements

1. Orders Table

```
CREATE TABLE ORDERS (  
    OrderID INTEGER      CONSTRAINT OrderID_PK PRIMARY KEY,  
    OrderType VARCHAR(10) CONSTRAINT OrderType_NOTNULL NOT NULL,  
    CustomerID NUMBER(10) CONSTRAINT CustomerID_NOTNULL NOT NULL,  
    EmployeeID INTEGER   CONSTRAINT EmployeeID_NOTNULL NOT NULL,  
    OrderTotal NUMBER(8,2) CONSTRAINT OrderTotal_NOTNULL NOT NULL,  
    LocationAddress VARCHAR2(100),  
    OrderDate DATE       CONSTRAINT OrderDate_NOTNULL NOT NULL,  
    PaymentID INTEGER  
);
```

2. Payment Table

```
CREATE TABLE PAYMENT (  
    PaymentID INTEGER      CONSTRAINT PaymentID_PK PRIMARY KEY,  
    OrderID INTEGER        CONSTRAINT OrderID_NOTNULL NOT NULL,  
    PaymentAmount NUMBER(5,2) CONSTRAINT PaymentAmount_NOTNULL NOT NULL,  
    PaymentType VARCHAR2(50) CONSTRAINT PaymentType_NOTNULL NOT NULL  
);
```

3. Recipe Table

```
CREATE TABLE RECIPE (  
    RecipeID INTEGER      CONSTRAINT RecipeID_PK PRIMARY KEY,  
    PizzaType VARCHAR2(50) CONSTRAINT PizzaType_NOTNULL NOT NULL,  
    PizzaSize VARCHAR2(50) CONSTRAINT PizzaSize_NOTNULL NOT NULL  
);
```

4. Menu Table

```
CREATE TABLE MENU_TABLE(  
    ItemID INTEGER      CONSTRAINT ItemID_PK PRIMARY KEY,  
    Price NUMBER(5,2)   CONSTRAINT Price_NOTNULL NOT NULL,  
    RecipeID INTEGER    CONSTRAINT RecipeID_NOTNULL NOT NULL,  
    ItemName VARCHAR2(100) CONSTRAINT ItemName_NOTNULL NOT NULL  
);
```

5. OrderDetails Table

```
CREATE TABLE ORDER_DETAILS(  
    OrderDetailsID INTEGER      CONSTRAINT OrderDetailsID_PK PRIMARY KEY,  
    SalePrice NUMBER(5,2)       CONSTRAINT SalePrice_NOTNULL NOT NULL,  
    Quantity INTEGER           CONSTRAINT Quantity_NOTNULL NOT NULL,
```

```

        OrderID INTEGER          CONSTRAINT ORDERS_DETAILS_OrderID_NOTNULL
        NOT NULL,
        ItemID INTEGER          CONSTRAINT ItemID_NOTNULL NOT NULL
    );

```

6. Customer Table

```

CREATE TABLE CUSTOMER(
    CustomerID NUMBER(10)    CONSTRAINT CustomerID_PK PRIMARY KEY,
    PhoneNumber NUMBER(10)   CONSTRAINT PhoneNumber_NOTNULL NOT NULL,
    FirstName VARCHAR2(100)  CONSTRAINT FirstName_NOTNULL NOT NULL,
    LastName VARCHAR2(100)   CONSTRAINT LastName_NOTNULL NOT NULL
);

```

7. Employee Table

```

CREATE TABLE EMPLOYEE(
    EmployeeID INTEGER       CONSTRAINT EmployeeID_PK PRIMARY KEY,
    FirstName VARCHAR2(100)  CONSTRAINT EMP_FirstName_NOTNULL NOT NULL,
    LastName VARCHAR2(100)  CONSTRAINT EMP_LastName_NOTNULL NOT NULL,
    Phone NUMBER(10)        CONSTRAINT EMP_PhoneNumber_NOTNULL NOT NULL,
    ManagerID INTEGER,
    Location VARCHAR2(100)   CONSTRAINT Location_NOTNULL NOT NULL,
    Address VARCHAR(100)     CONSTRAINT Address_NOTNULL NOT NULL
);

```

8. Location Table

```

CREATE TABLE LOCATION(
    Address VARCHAR2(100)    CONSTRAINT Address_PK PRIMARY KEY,
    EmployeeID INTEGER       CONSTRAINT EmployeeID_NOTNULL NOT NULL
);

```

9. Foreign Keys

```

ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDERDETAILS_MENU
FOREIGN KEY (ItemID) REFERENCES MENU_TABLE(ItemID);

```

```

ALTER TABLE MENU_TABLE ADD CONSTRAINT FK_MENU_RECIPE FOREIGN
KEY (RecipeID) REFERENCES RECIPE(RecipeID);

```

```

ALTER TABLE EMPLOYEE
ADD CONSTRAINT FK_EMPLOYEE_MANAGER FOREIGN KEY (ManagerID)
REFERENCES EMPLOYEE(EmployeeID);

```

```

ALTER TABLE LOCATION ADD CONSTRAINT FK_LOCATION_EMPLOYEE
FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEE(EmployeeID);

```

```

ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_CUSTOMER FOREIGN
KEY (CustomerID) REFERENCES CUSTOMER(CustomerID);

```

```
ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_EMPLOYEE FOREIGN  
KEY (EmployeeID) REFERENCES EMPLOYEE(EmployeeID);
```

```
ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_PAYMENT FOREIGN  
KEY (PaymentID) REFERENCES PAYMENT(PaymentID);
```

```
ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDERDETAILS_ORDERS  
FOREIGN KEY (OrderID) REFERENCES ORDERS(OrderID);
```

```
ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_EMPLOYEE_LOCATION  
FOREIGN KEY (Address) REFERENCES LOCATION(Address);
```

```
ALTER TABLE PAYMENT ADD CONSTRAINT FK_PAYMENT_ORDERS FOREIGN  
KEY (OrderID) REFERENCES ORDERS(OrderID);
```

B. Oracle DML statements: INSERT statements

1. Orders Table

```
INSERT INTO ORDERS VALUES (1, 'Pickup', 101, 201, 25.50, '123 Main St', DATE  
'2026-04-01', 301);
```

```
INSERT INTO ORDERS VALUES (2, 'Delivery', 102, 202, 40.75, '456 Oak Ave', DATE  
'2026-04-02', 302);
```

```
INSERT INTO ORDERS VALUES (3, 'Pickup', 103, 203, 15.25, '789 Pine Rd', DATE  
'2026-04-03', 303);
```

```
INSERT INTO ORDERS VALUES (4, 'Delivery', 104, 204, 60.00, '321 Maple St', DATE  
'2026-04-04', 304);
```

```
INSERT INTO ORDERS VALUES (5, 'Pickup', 105, 205, 22.10, '654 Elm St', DATE  
'2026-04-05', 305);
```

```
INSERT INTO ORDERS VALUES (6, 'Delivery', 106, 206, 35.80, '987 Cedar Ave',  
DATE '2026-04-06', 306);
```

```
INSERT INTO ORDERS VALUES (7, 'Pickup', 107, 207, 18.99, '147 Birch Rd', DATE  
'2026-04-07', 307);
```

```
INSERT INTO ORDERS VALUES (8, 'Delivery', 108, 208, 55.45, '258 Walnut St', DATE  
'2026-04-08', 308);
```

```
INSERT INTO ORDERS VALUES (9, 'Pickup', 109, 209, 27.30, '369 Cherry Ave', DATE  
'2026-04-09', 309);
```

```
INSERT INTO ORDERS VALUES (10, 'Delivery', 110, 210, 48.60, '159 Spruce Rd',  
DATE '2026-04-10', 310);
```

```
COMMIT;
```

2. Payment Table

```
INSERT INTO PAYMENT VALUES (301, 1, 25.50, 'Credit Card');
```

```
INSERT INTO PAYMENT VALUES (302, 2, 40.75, 'Debit Card');
```

```
INSERT INTO PAYMENT VALUES (303, 3, 15.25, 'Cash');
```

```
INSERT INTO PAYMENT VALUES (304, 4, 60.00, 'Credit Card');
```

```
INSERT INTO PAYMENT VALUES (305, 5, 22.10, 'Mobile Pay');
```

```
INSERT INTO PAYMENT VALUES (306, 6, 35.80, 'Debit Card');
```

```
INSERT INTO PAYMENT VALUES (307, 7, 18.99, 'Cash');
```

```
INSERT INTO PAYMENT VALUES (308, 8, 55.45, 'Credit Card');
```

```
INSERT INTO PAYMENT VALUES (309, 9, 27.30, 'Mobile Pay');
```

```
INSERT INTO PAYMENT VALUES (310, 10, 48.60, 'Debit Card');
```

3. Recipe Table

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (1, 'Pepperoni',  
'Small');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (2, 'Pepperoni',  
'Medium');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (3, 'Pepperoni',  
'Large');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (4, 'Cheese',  
'Small');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (5, 'Cheese',  
'Large');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (6, 'Meat Lovers',  
'Medium');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (7, 'Meat Lovers', 'Large');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (8, 'Veggie', 'Small');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (9, 'Veggie', 'Medium');
```

```
INSERT INTO RECIPE (RecipeID, PizzaType, PizzaSize) VALUES (10, 'Supreme', 'Large');
```

```
COMMIT;
```

4. Menu Table

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (501, 12.99, 1, 'Small Pepperoni Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (502, 15.49, 2, 'Medium Pepperoni Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (503, 18.95, 3, 'Large Pepperoni Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (504, 10.00, 4, 'Small Cheese Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (505, 14.50, 5, 'Large Cheese Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (506, 16.99, 6, 'Medium Meat Lovers');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (507, 21.00, 7, 'Large Meat Lovers');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (508, 11.25, 8, 'Small Veggie Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (509, 14.75, 9, 'Medium Veggie Pizza');
```

```
INSERT INTO MENU_TABLE (ItemID, Price, RecipeID, ItemName) VALUES (510, 19.99, 10, 'Large Supreme Pizza');
```

COMMIT;

5. OrderDetails Table

INSERT INTO ORDER_DETAILS VALUES (1, 12.99, 2, 1, 501);

INSERT INTO ORDER_DETAILS VALUES (2, 8.50, 1, 2, 502);

INSERT INTO ORDER_DETAILS VALUES (3, 15.75, 3, 3, 503);

INSERT INTO ORDER_DETAILS VALUES (4, 9.99, 2, 4, 504);

INSERT INTO ORDER_DETAILS VALUES (5, 20.00, 1, 5, 505);

INSERT INTO ORDER_DETAILS VALUES (6, 7.25, 4, 6, 506);

INSERT INTO ORDER_DETAILS VALUES (7, 11.50, 2, 7, 507);

INSERT INTO ORDER_DETAILS VALUES (8, 13.80, 1, 8, 508);

INSERT INTO ORDER_DETAILS VALUES (9, 6.99, 5, 9, 509);

INSERT INTO ORDER_DETAILS VALUES (10, 18.40, 2, 10, 510);

COMMIT;

6. Customer Table

INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (4, 5135551004, 'Olivia', 'Brown');

INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (5, 5135551005, 'Noah', 'Jones');

INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (6, 5135551006, 'Ava', 'Garcia');

INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (7, 5135551007, 'Ethan', 'Miller');

INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (8, 5135551008, 'Sophia', 'Davis');


```
INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (9, 5135551009, 'Mason', 'Rodriguez');
```

```
INSERT INTO CUSTOMER (CustomerID, PhoneNumber, FirstName, LastName)
VALUES (10, 5135551010, 'Isabella', 'Martinez');
```

```
COMMIT;
```

7. Employee Table

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (101, 'James', 'Wilson', 5135552001, NULL, 'Cincinnati',
'123 Maple St');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (102, 'Sarah', 'Miller', 5135552002, 101, 'Cincinnati', '456
Oak Ave');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (103, 'Robert', 'Taylor', 6145553003, 101, 'Columbus', '789
Pine Rd');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (104, 'Linda', 'Anderson', 6145553004, 103, 'Columbus',
'321 Elm St');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (105, 'Michael', 'Thomas', 2165554005, 101, 'Cleveland',
'654 Birch Ln');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (106, 'Karen', 'White', 2165554006, 105, 'Cleveland', '987
Cedar Blvd');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (107, 'David', 'Harris', 5135552007, 102, 'Cincinnati', '159
Walnut Ct');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (108, 'Susan', 'Martin', 3305555008, 105, 'Akron', '753
Willow Dr');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,
Location, Address) VALUES (109, 'Steven', 'Thompson', 3305555009, 108, 'Akron', '852
Spruce St');
```

```
INSERT INTO EMPLOYEE (EmployeeID, FirstName, LastName, Phone, ManagerID,  
Location, Address) VALUES (110, 'Emily', 'Garcia', 4195556010, 101, 'Toledo', '951  
Aspen Way');
```

```
COMMIT;
```

8. Location Table

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('123 Maple St', 101);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('456 Oak Ave', 102);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('789 Pine Rd', 103);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('321 Elm St', 104);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('654 Birch Ln', 105);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('987 Cedar Blvd', 106);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('159 Walnut Ct', 107);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('753 Willow Dr', 108);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('852 Spruce St', 109);
```

```
INSERT INTO LOCATION (Address, EmployeeID) VALUES ('951 Aspen Way', 110);
```

```
COMMIT;
```

Example #1

The first example of an operational transaction this database supports is the creation of a new customer order. This transaction occurs when a customer places a pizza order, either in-store, online, or by phone.

The first step in this transaction is to validate or create a record in the Customers table. Then, insert a new row into the Orders table. Next we assign the OrderID, link the CustomerID and EmployeeID, and set OrderType, Date, and the LocationAddress. For each item ordered, we

insert rows into OrderDetails (which Include ItemID, Quantity, and Price). Then, we calculate and update OrderTotal in Orders. From there, we record the PaymentID for tracking payment.

This is the primary revenue-generating process, ensuring accurate order tracking and linking customers, employees, and menu items.

Example #2

The second example of an operational transaction this database supports is Assigning Employees to Orders. This transaction tracks which employee is responsible for preparing or handling an order. The first step in this transaction is to select an available employee from the Employees table. Then, update the Orders table to set the EmployeeID. Then we need to ensure that the employee belongs to the correct Location.

This transaction supports accountability and performance tracking, and helps managers evaluate employee workload and efficiency.

Example #3

The third example of an operational transaction this database supports is to update menu offerings and their associated Recipes. The two associated tables are Recipe and MenuItem. The first step in this transaction is to insert in the Recipe ItemType and ItemSize. This is a choice between Cheese or Pepperoni and a small or large pizza. Next, you insert in the MenuItem table and link it to RecipeID while setting the price of each item. Once we do that we are able to make sure that all menu items are connected to OrderDetails.

This transaction supports accurate price and consistency in food preparation. That way if we were to change the price of a menu item we would be able to do so easily.